SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1702**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I M. Santosh (192521007) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Answer:

CSP formulation: Variables: D1, D2, D3 (each doctor). Domain of each: {Morning (M), Afternoon (A), Night (N)}. Constraints: C1: D1 ≠ N. C2: shift(D2) < shift(D3) in the order M < A < N. C3: all-different (one doctor per shift, each doctor exactly one shift). C4: D3 ≠ M. Variable order: D1, D2, D3; value order: M, A, N.

Step	Assignment tried	Constraint check	Result
1	D1 = M	C1 OK (not Night)	Accept
2	D1=M, D2 = M	C3 fails (Morning already taken)	Reject
3	D1=M, D2 = A	C3 OK; C2 not yet testable	Accept
4	D1=M, D2=A, D3 = M	C3 fails (taken); C4 fails (D3 ≠ M)	Reject
5	D1=M, D2=A, D3 = A	C3 fails (Afternoon taken by D2)	Reject
6	D1=M, D2=A, D3 = N	C3 OK; C4 OK; C2: A < N OK	SOLUTION 1
—	Continue to enumerate all solutions: backtrack to D2		
7	D1=M, D2 = N	C3 OK	Accept
8	D1=M, D2=N, D3 = M	C3 fails; C4 fails	Reject → backtrack
9	D1=M, D2=N, D3 = A	C2 fails (N is not before A)	Reject
10	D1=M, D2=N, D3 = N	C3 fails (taken)	Reject → backtrack to D1
11	D1 = A	C1 OK	Accept
12	D1=A, D2 = M	C3 OK	Accept
13	D1=A, D2=M, D3 = M / A	C3 fails (both taken)	Reject
14	D1=A, D2=M, D3 = N	C3, C4 OK; C2: M < N OK	SOLUTION 2
15	D1=A, D2 = A	C3 fails	Reject
16	D1=A, D2 = N, D3 = M / A / N	M, A: C4 or C2 fails; N: C3 fails	Reject → backtrack to D1

17	D1 = N	C1 fails (D1 cannot work Night)	Reject – search ends

Final valid schedule (first solution found by backtracking): Morning → D1, Afternoon → D2, Night → D3.

Note: the exhaustive search shows exactly one other valid schedule: Morning → D2, Afternoon → D1, Night → D3. Both satisfy all five constraints.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right
ii. Cost of each move = 1
iii. Cannot pass through obstacles
iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

Answer:

Assumed grid (the question does not include a figure): 5×5 grid, cells written as (row, column) with row 1 at the top. Start S = (1,1), Goal G = (5,5). Obstacles (X) = (2,2), (2,3), (2,4), (4,2), (4,3), (4,4).

	c1	**c2**	**c3**	**c4**	**c5**
r1	S				
r2		X	X	X	
r3					
r4		X	X	X	
r5					G

Method: BFS expands nodes in FIFO (queue) order, i.e. in increasing number of moves. Since every move costs 1, the first time BFS reaches G it has found the cheapest path. Neighbours are generated in the order Up, Down, Left, Right; visited cells are not re-added. The Manhattan heuristic $h(n) = |r - 5| + |c - 5|$ is shown beside each node for reference: BFS itself does not use it (that would be A*), so the queue is ordered by arrival (equivalently, by g), not by f = g + h.

Step	**Expand (g, h)**	**Nodes added**	**Queue after expansion (front → back)**
1	(1,1) g=0, h=8	(2,1), (1,2)	(2,1), (1,2)
2	(2,1) g=1, h=7	(3,1)	(1,2), (3,1)
3	(1,2) g=1, h=7	(1,3)	(3,1), (1,3)
4	(3,1) g=2, h=6	(4,1), (3,2)	(1,3), (4,1), (3,2)
5	(1,3) g=2, h=6	(1,4)	(4,1), (3,2), (1,4)
6	(4,1) g=3, h=5	(5,1)	(3,2), (1,4), (5,1)
7	(3,2) g=3, h=5	(3,3)	(1,4), (5,1), (3,3)

8	(1,4) g=3, h=5	(1,5)	(5,1), (3,3), (1,5)
9	(5,1) g=4, h=4	(5,2)	(3,3), (1,5), (5,2)
10	(3,3) g=4, h=4	(3,4)	(1,5), (5,2), (3,4)
11	(1,5) g=4, h=4	(2,5)	(5,2), (3,4), (2,5)
12	(5,2) g=5, h=3	(5,3)	(3,4), (2,5), (5,3)
13	(3,4) g=5, h=3	(3,5)	(2,5), (5,3), (3,5)
14	(2,5) g=5, h=3	none (neighbours visited)	(5,3), (3,5)
15	(5,3) g=6, h=2	(5,4)	(3,5), (5,4)
16	(3,5) g=6, h=2	(4,5)	(5,4), (4,5)
17	(5,4) g=7, h=1	(5,5) = G	(4,5), (5,5)
18	(4,5) g=7, h=1	none	(5,5)
19	(5,5) g=8, h=0	Goal test passed	Stop

Optimal path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5), i.e. Down ×4 then Right ×4.

Optimal cost: 8 moves × 1 = 8. This equals h(S) = 8, so the heuristic is exactly tight here and the path is optimal. 18 nodes were expanded before the goal.

Remark: with the Manhattan heuristic in a priority queue ordered by f = g + h (A*), the search would head directly to G and expand far fewer nodes, but the path and cost (8) would be the same.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

Answer (a): Effect of the constraints on decision-making

- **Movement in 4 directions, cost 1:** the branching factor is at most 4 and the world is a grid graph, so path cost is just steps plus risk penalties; this suits a cost-based search.
- **Blocked cells:** the robot must never plan through an obstacle. When it discovers one, the current plan becomes invalid and it must re-plan, so the decision depends on newly sensed information.

- **Limited sensing (adjacent cells only):** the environment is partially observable; the robot cannot plan a full path in advance and must keep an internal map (belief state), choosing the next move from what it currently knows.
- **Risky zones (+2 cost, total 3):** the cheapest route is no longer the shortest one. A risky cell costs 3, so a detour of up to 2 extra safe steps is preferable to crossing one risky cell. This forces cost-sensitive search rather than BFS.
- **Minimise cost and stay safe:** two objectives: low path cost and low risk. Safety is built into the cost function (penalty), and cells known to be blocked/dangerous are excluded; this prioritises safety over speed.
- **Dynamic knowledge update (online search):** the robot interleaves planning and acting: sense → update map → re-plan → move. Decisions are made one step at a time and revised continuously.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

Answer (b): Solution approach – Online Uniform Cost Search (re-planning agent)

- **Step 1 – Model:** State = robot cell; known map M (free, blocked, risky, unknown). Step cost $c(n \to n') = 1$ for a normal cell and $1 + 2 = 3$ for entering a risky cell. Unknown cells are treated as free with cost 1 (optimistic) until sensed.
- **Step 2 – Sense:** at the current cell read the 4 adjacent cells (blocked / free / risky) and update M.
- **Step 3 – Plan:** run Uniform Cost Search on M from the current cell to G: keep a priority queue ordered by g(n), always expand the cheapest node, ignore blocked cells, and stop when G is popped. This gives the least-cost path on the known map.
- **Step 4 – Act:** execute only the first move of this path (or until new information arrives) and add the cost to the running total.
- **Step 5 – Re-plan:** after every move, sensing may reveal a new obstacle or risky cell. If the remaining path is affected, go back to Step 3 and run UCS again from the new position. If the robot is boxed in, backtrack to the last junction with unexplored options.
- **Step 6 – Terminate:** stop when the robot reaches G (survivor found) or report failure if no path exists on the known map.

```
function ONLINE-UCS-AGENT(percept) returns action
  persistent: map M, position p, goal G, path
  M <- UPDATE-MAP(M, percept)          // adjacent cells
  if path is empty or path is blocked in M then
      path <- UCS(M, p, G)             // cost: 1, risky 3
      if path = failure then return STOP
  a <- FIRST(path); path <- REST(path)
  return a

function UCS(M, start, G)
  frontier <- priority queue {start : 0}; explored <- {}
  while frontier not empty do
     n <- POP-LOWEST-g(frontier)
     if n = G then return PATH(n)
     add n to explored
     for each neighbour m of n in M, m not blocked do
         g2 <- g(n) + (3 if risky(m) else 1)
         if m not in frontier/explored or g2 < g(m) then
             INSERT/UPDATE(frontier, m, g2)
  return failure
```

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

Answer (c): AI concepts applied

- **Intelligent agent:** the robot is a goal-based, utility-oriented agent. Percepts = adjacent cell contents and its position; actions = Up, Down, Left, Right (actuators); it maps percepts to actions through its sense–update–plan–act loop and keeps an internal model (the known map), so it is a model-based agent.
- **Rationality:** a rational agent picks the action expected to maximise its performance measure given its percept history and knowledge. Here, the performance measure is reaching G with the minimum total cost (steps + risk penalties) and with safety. The robot is rational because it always chooses the next move on the cheapest known path and re-plans when new percepts change what is best. Rationality does not mean omniscience: it acts on what it can sense.
- **Environment type:** partially observable (only adjacent cells sensed), deterministic in movement (a move succeeds unless blocked), sequential (each move affects future costs), static between moves (the map does not change while the robot plans, though new parts are discovered), discrete (grid cells and 4 actions), single-agent, and initially unknown (exploration needed). These properties are exactly why an online, re-planning search agent is needed.

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Answer (d): Justification

- Uniform Cost Search is optimal and complete for non-negative step costs, so on the known map it always returns the cheapest safe route, which fits the "minimise total cost" requirement. BFS would ignore the +2 risk penalty and DFS gives no cost guarantee.
- A* could be used with an admissible heuristic (Manhattan distance, which never overestimates since each step costs at least 1) to expand fewer nodes, but UCS needs no heuristic and works even when risky costs make the heuristic less informative.
- An online (interleaved) agent is required because the environment is partially observable: an offline planner would need the full map in advance, which the robot does not have. Re-planning after each new percept lets the robot handle hidden obstacles safely.
- Risk is encoded as extra cost, so safety and efficiency are balanced inside one search rather than handled by separate rules.
- Overall, online UCS with a continuously updated map is optimal on what is known, safe, and adaptive to new information, making it the most suitable strategy under the stated constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided